Kodewala Academy(1st June, 2026) - Java Backend Test # 6

Daksh Arya
July 31, 2026, 9:56 a.m.

You've survived the test. The results will arrive soon. 😎

Your score: 88% 26.25/30
Duration: 0:14:47

1. A production application receives around 2 million requests per day. The developer writes: — 1 / 1 point

```
public class UserService {
    public String getMessage(String name) {
        String msg = "";
        msg += "Hello ";
        msg += name;
        msg += ", Welcome";
        return msg;
    }
}
```

The method is executed millions of times.

What is the best improvement?

Your Answer: ✔ Correct

○ Replace String with StringBuffer
✔ ◉ Replace String with StringBuilder
○ Make msg final
○ No change required

2. Your company has a framework used by 150 services. — 1 / 1 point

```
interface PaymentService{
    void pay();
}
```

A new feature requires adding logging without breaking existing implementations.

What should you do?

Your Answer: ✔ Correct

○ Add another abstract method
✔ ◉ Add default method
○ Add constructor
○ onvert interface into abstract class

3. A company stores user sessions in Redis using Java Serialization. — 1 / 1 point

```
class User implements Serializable{
    String name;
    transient String password;
}
```

After deserialization, what happens?

Your Answer: ✔ Correct

○ password is restored
✔ ◉ password becomes null
○ Compilation error
○ Runtime exception

4. — -0.25 / 1 point

```
class Parent{
    void print(Object o){
        System.out.println("Parent");
    }
}
class Child extends Parent{
    void print(String s){
        System.out.println("Child");
    }
}
Parent p=new Child();
p.print("Java");
```

Output?

Your Answer: ✘ Incorrect

✔ ○ Parent
✘ ◉ Child
○ Compilation Error
○ Runtime Error

5. Your team wants to prevent unauthorized classes from being cached. — 1 / 1 point

```
interface Cacheable{
}
```

Only classes implementing Cacheable are accepted.

Why use a marker interface instead of a normal interface with methods?

Your Answer: ✔ Correct

○ Better performance
✔ ◉ To mark eligibility without forcing implementation
○ JVM requirement
○ To reduce memory

6. A banking application has — 1 / 1 point

```
final class EncryptionUtil
{
}
```

Why is the class declared final?

Your Answer: ✔ Correct

○ Better memory
○ Faster execution
○ Required for serialization
✔ ◉ Prevent inheritance and modification of security logic

7. — 1 / 1 point

```
class Parent
{
    public static void show(){
        System.out.println("Parent");
    }
}
class Child extends Parent
{
    public static void show()
  {
        System.out.println("Child");
    }
}
Parent p = new Child();
p.show();
```

Your Answer: ✔ Correct

✔ ◉ Parent
○ Child
○ Runtime Error
○ Compile Error

8. — 1 / 1 point

```
interface A{
    default void show(){
        System.out.println("A");
    }
}
interface B{
    default void show(){
        System.out.println("B");
    }
}
class Test implements A,B{
}
```

What happens?

Your Answer: ✔ Correct

○ Prints A
○ Prints B
○ Runtime Error
✔ ◉ Compile Error

9. A Serializable class was deployed. — 1 / 1 point

Later a developer changes the class.

Old serialized files can no longer be read.

What should have been added?

Your Answer: ✔ Correct

○ transient
✔ ◉ serialVersionUID
○ final
○ Serializable

10. Why were private methods added to interfaces? — 1 / 1 point

Your Answer: ✔ Correct

○ To allow object creation
✔ ◉ To reuse code between default methods
○ For inheritance
○ To improve serialization

11. — 1 / 1 point

```
class Parent{
    final void show()
  {
        System.out.println("Parent");
    }
}
class Child extends Parent{
    void show(){
        System.out.println("Child");
    }
}
```

Your Answer: ✔ Correct

○ Parent
○ Child
○ Runtime Error
✔ ◉ Compilation Error

12. A banking application uses String for passwords. — 1 / 1 point

Why?

Your Answer: ✔ Correct

○ Faster than StringBuilder
✔ ◉ Immutable, safer for security
○ Consumes less memory
○ Allows modification

13. — 1 / 1 point

```
class Employee
{


}
public class Test {
    public static void main(String[] args) {
        Employee emp = new Employee();
        System.out.println(emp instanceof Object);
    }
}
```

What is the output?

Your Answer: ✔ Correct

○ false
✔ ◉ true
○ Compilation Error
○ Runtime Exception

14. Which statement about the Object class is correct? — 1 / 1 point

Your Answer: ✔ Correct

○ Only abstract classes extend Object
○ Only interfaces extend Object
✔ ◉ Every class in Java directly or indirectly extends Object
○ Only user-defined classes extend Object

15. Which of the following is TRUE about the String class? — 1 / 1 point

Your Answer: ✔ Correct

○ String objects are mutable.
✔ ◉ String objects are immutable.
○ String cannot be compared using equals().
○ String is not a class.

16. Why is StringBuffer preferred over StringBuilder in a multi-threaded environment? — 1 / 1 point

Your Answer: ✔ Correct

○ It is immutable.
○ It consumes less memory.
✔ ◉ It is synchronized and thread-safe.
○ It is faster than StringBuilder.

17. Which statement about StringBuilder is correct? — 1 / 1 point

Your Answer: ✔ Correct

○ It is immutable.
○ It is synchronized.
✔ ◉ It is mutable and generally faster than StringBuffer in a single-threaded environment.
○ It stores String literals in the String Pool.

18. Which of the following methods can be added to an interface in Java 8 without breaking existing implementations? — 1 / 1 point

Your Answer: ✔ Correct

○ Abstract method
✔ ◉ Default method
○ Constructor
○ Protected method

19. Which statement is TRUE about an abstract class? — 1 / 1 point

Your Answer: ✔ Correct

○ It cannot have constructors.
○ It cannot have concrete methods.
✔ ◉ It cannot be instantiated.
○ It supports multiple inheritance.

20. Which of the following is a marker interface? — 1 / 1 point

Your Answer: ✔ Correct

○ Comparable
○ Runnable
✔ ◉ Cloneable
○ Callable

21. What is the default behavior of Object.clone()? — 1 / 1 point

Your Answer: ✔ Correct

○ Deep Copy
✔ ◉ Shallow Copy
○ No Copy
○ Serialization

22. Which block is executed only once when a class is loaded into the JVM? — 1 / 1 point

Your Answer: ✔ Correct

○ Constructor
○ Instance Initialization Block
✔ ◉ Static Initialization Block

23. Which of the following is NOT required for method overloading? — 1 / 1 point

Your Answer: ✔ Correct

○ Different parameter list
○ Same method name
○ Same class
✔ ◉ Different return type only

24. Which method compares the contents of two String objects? — 1 / 1 point

Your Answer: ✔ Correct

○ ==
○ compareTo()
✔ ◉ equals()
○ hashCode()

25. What is a Covariant Return Type in Java? — 1 / 1 point

Your Answer: ✔ Correct

○ A method can return multiple values.
✔ ◉ An overridden method can return a subclass of the original return type.
○ An overloaded method can have different return types.
○ A method can return only primitive data types.

26. — 1 / 1 point

```
class Employee {
    static int companyCount = 0;
    Employee() {
        companyCount++;
    }
}
public class Test {
    public static void main(String[] args) {
        new Employee();
        new Employee();
        new Employee();
        System.out.println(Employee.companyCount);
    }
}
```

Your Answer: ✔ Correct

○ 1
○ 2
✔ ◉ 3
○ 0

27. — 1 / 1 point

```
class Employee {
    int id = 100;
}
public class Test {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee();
        e1.id = 200;
        System.out.println(e2.id);
    }
}
```

Your Answer: ✔ Correct

✔ ◉ 100
○ 200
○ 0
○ Compilation Error

28. A company wants to maintain the total number of active users across the application. — -0.25 / 1 point
Which variable should be used?

Your Answer: ✘ Incorrect

✘ ◉ Instance variable
○ Local variable
✔ ○ Static variable
○ Constructor parameter

29. Where are instance variables stored? — 1 / 1 point

Your Answer: ✔ Correct

○ Method Area
✔ ◉ Heap Memory
○ Stack Memory

30. An e-commerce application contains: — -0.25 / 1 point

- Product Name
- Product Price
- GST Percentage (same for every product)

Which declaration is the best design?

Your Answer: ✘ Incorrect

○ double gst;
✔ ○ static double gst;
✘ ◉ final double gst
○ volatile double gst;

Logout